

Entity Framework Part 1

Advanced C# Programming - NET .022

Prepared By: Rupinder Sandhu
Rupinder.sandhu@humber.ca



**WE ARE
HUMBER**

Agenda

Entity Framework

- Object Relational Mismatch
- Architecture
- LinqToSql
- DbContext
 - DbSet<T>
 - Entry<T>
 - SaveChanges

Object to Relational Impedance Mismatch

Objects	Databases
Built using OOP principles	Built using relational algebra
Use inheritance and aggregation (collections)	Requires data normalization
Link with references	Link with foreign keys
Identified by memory location	Identified by primary key
Use data types defined by runtime	Use datatypes defined by database
Can hold data in lists and dictionaries	Can hold data in tuples
Not transactional	Heavily transactional

LINQ to SQL

- Introduces an ORM into the .NET framework
 - System.Data.Linq.dll
- Standard LINQ query operators still apply!
 - Filtering, grouping, sorting, joining ...
- Generates parameterized SQL to execute on the database server

What is the ADO.NET Entity Framework?

- Entity Framework is a type of ORM
- A development platform that provides a layer of abstraction on top of the relational model
- It isolates the object model of the application from the way the data is actually stored in the relational store.

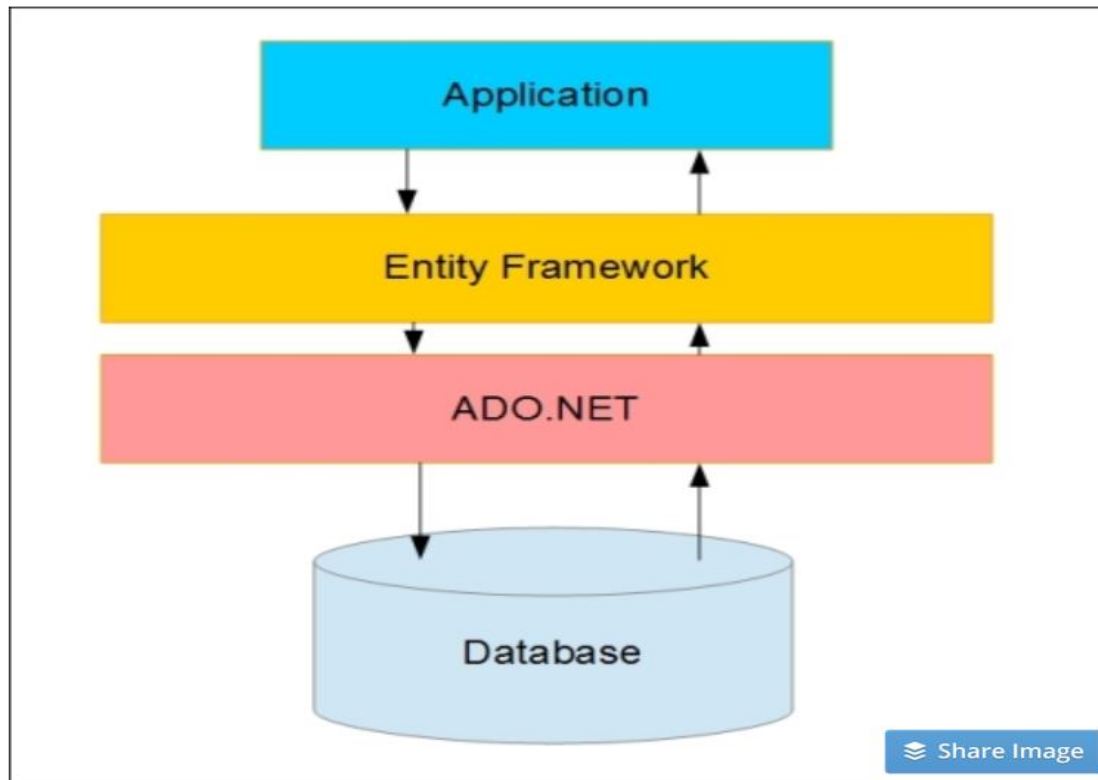
Features and benefits

- It provides support for an increased level of abstraction on top of the underlying data store
- It provides support for extensibility and seamless querying of data using Entity SQL and LINQ
- It is a flexible schema that can be used to store the mapping information

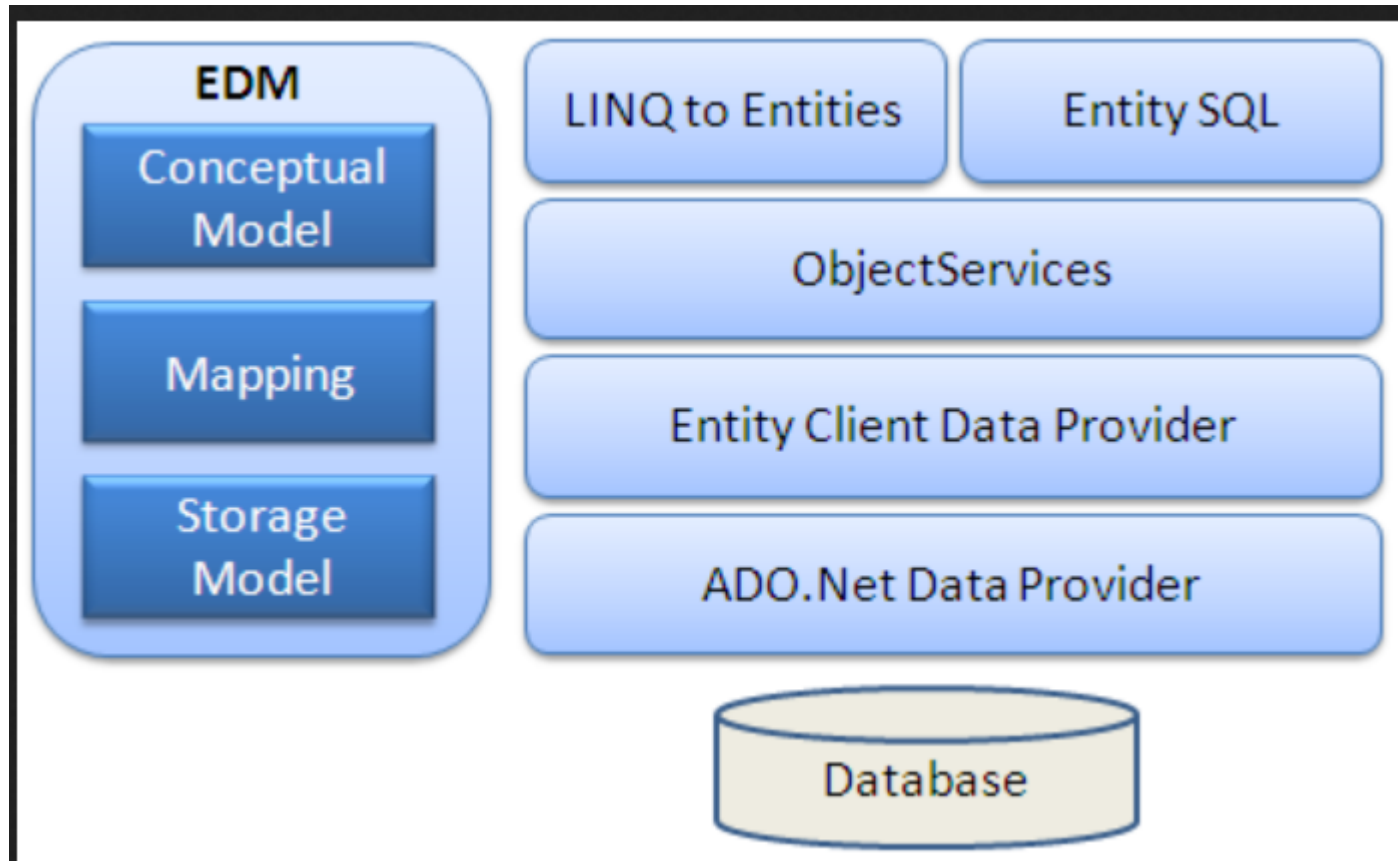
Features and benefits (cont.)

- There is a reduction in the amount of code needed to write data access code in your applications
- It contains a powerful Object Services Layer
- It provides support for a full text-based query language
- It provides support for persistence ignorance
- It provides support for lazy loading

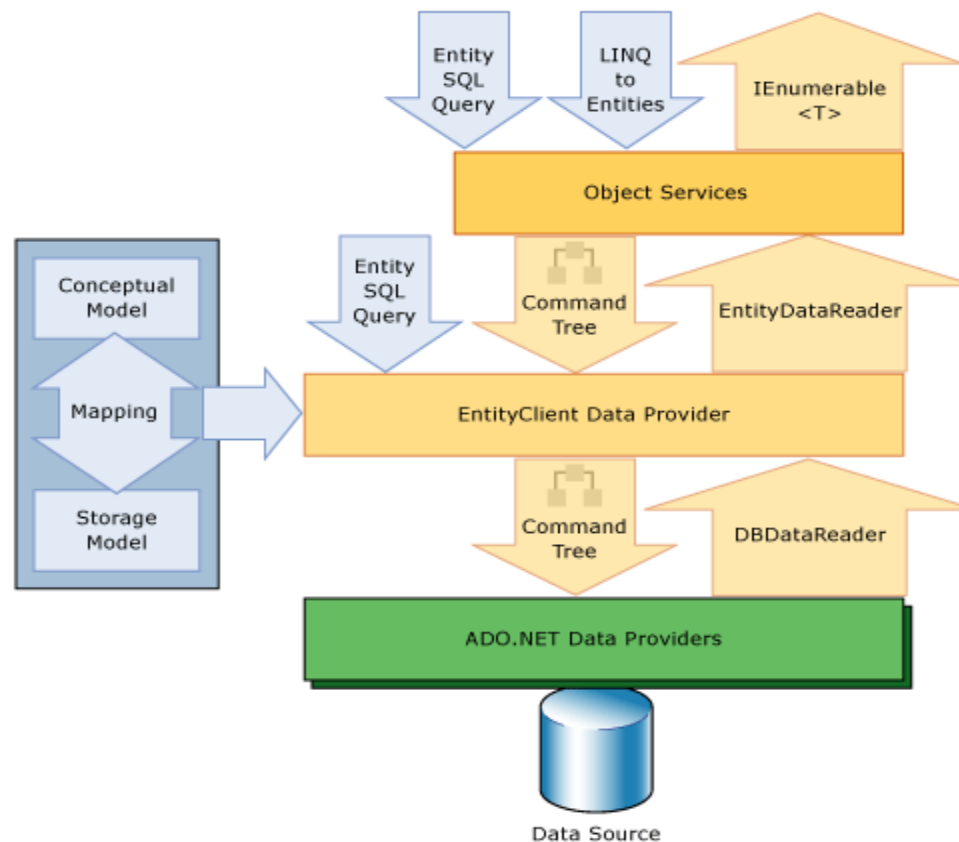
Entity Framework Architecture



Entity Framework Architecture (cont.)



Entity Framework Architecture (cont.)



EDM (Entity Data Model)

- EDM consists of three main parts - Conceptual model, Mapping and Storage model.
 - Conceptual Model: The conceptual model contains the model classes and their relationships. This will be independent from your database table design.

EDM (cont.)

- Storage Model: Storage model is the database design model which includes tables, views, stored procedures, and their relationships and keys.
- Mapping: Mapping consists of information about how the conceptual model is mapped to the storage model.

LINQ to Entities

- LINQ to Entities is a query language used to write queries against the object model. It returns entities, which are defined in the conceptual model. You can use your LINQ skills here.

Entity SQL

- Entity SQL is another query language just like LINQ to Entities. However, it is a little more difficult than L2E and the developer will have to learn it separately.

Entity Client Data Provider

- The main responsibility of this layer is to convert L2E or Entity SQL queries into a SQL query which is understood by the underlying database. It communicates with the ADO.Net data provider which in turn sends or retrieves data from the database.

ADO.Net Data Provider

- This layer communicates with the database using standard ADO.Net.

DbContext

- The DbContext is the main conduit by which you retrieve objects from the database and submit changes back to the database.
- The DbContext class is the base class for the data context in your custom application.
- DbContext allows us to:
 - Configure model & relationship
 - Querying database
 - Saving data to the database

Defining a DbContext derived class

- Define a class that inherits from DbContext
- Exposes DbSet properties that represent collections of the specified entities in the context.

DbContext.DbSet<T>

- Represents an entity set that is used for create, read, update, and delete operations.
- You can get the reference of DbSet by using DbContext

DbContext.OnModelCreating

- This method is called when the model for a derived context has been initialized, but before the model has been locked down and used to initialize the context.
- The default implementation of this method does nothing, but it can be overridden in a derived class such that the model can be further configured before it is locked down.

Entity Framework Entity States

- An entity can be in one of five states as defined by the EntityState enumeration. These states are:
 - Added: the entity is being tracked by the context but does not yet exist in the database
 - Unchanged: the entity is being tracked by the context and exists in the database, and its property values have not changed from the values in the database

Entity Framework Entity States (cont.)

- **Modified:** the entity is being tracked by the context and exists in the database, and some or all of its property values have been modified
- **Deleted:** the entity is being tracked by the context and exists in the database, but has been marked for deletion from the database the next time `SaveChanges` is called
- **Detached:** the entity is not being tracked by the context

DbContext.Entry<T>

- Gets a DbEntityEntry object for the given entity providing access to information about the entity and the ability to perform actions on the entity.

DbContext.SaveChanges

- Saves all changes made in this context to the underlying database.
- When you call `DbContext.SaveChanges()`, it performs insert, update or delete operation on the database based on `EntityState` of the entities.

THANK YOU.



WE ARE

HUMBER